

Gators: a Python package for high-performance machine learning preprocessing and feature engineering on Polars, with portable ONNX export

Charles Poli¹ and Shreekanthadatta Eligar¹

¹ PayPal, Inc., San Jose, CA, USA ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: ↗

Submitted: 15 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/))

Summary

Data preprocessing and feature engineering are executed far more often than model training: every serving request re-runs the same transformations that were fit once during training. Gators is an open-source Python library that provides 107 transformers, organized into 10 categories (data cleaning, outlier clipping, categorical encoding – including CatBoost-style target encoding (Prokhorenkova et al., 2018) – numeric/string/datetime feature generation, missing-value imputation, discretization, scaling, and feature selection), implemented on top of the Polars DataFrame engine (Vink & Polars Contributors, 2020) and exposed through a scikit-learn-compatible fit/transform API (Pedregosa et al., 2011). A fitted Gators pipeline can be chained with `gators.pipeline.Pipeline` and exported, as a single object, to a validated ONNX graph (ONNX Contributors, 2019a) for runtime-agnostic, language-agnostic inference — without hand-maintaining a second implementation of the same preprocessing logic for production. The library is tested with 2,323 unit and ONNX-conversion tests and ships a reproducible benchmark suite comparing it against scikit-learn and feature-engine (Galli, 2021) across row counts, feature counts, missingness, cardinality, and thread configurations.

Statement of need

Preprocessing code is frequently treated as disposable glue rather than a first-class part of a machine learning system, despite being both a major source of compute cost on large tabular datasets and a major source of production failures. Sculley et al. (Sculley et al., 2015) identify data-pipeline glue code as one of the largest sources of technical debt in ML systems, and Breck et al. (Breck et al., 2017) note that *training/serving skew* — a mismatch between how a feature is computed at training time versus serving time — is one of the most common causes of silent production degradation. The standard mitigations are either (a) maintaining two implementations of the same feature logic (a notebook version in pandas and a hand-written production version in a second language), and testing the two for parity, or (b) tying the preprocessing logic to a single serving runtime. Neither is satisfying: (a) is a recurring maintenance and correctness burden, and (b) trades away portability. Gators targets researchers and practitioners who need (i) a wide, consistent catalogue of preprocessing operations that composes with existing scikit-learn tooling, (ii) an implementation fast enough that preprocessing is not the bottleneck in a research iteration loop, and (iii) a way to take a fitted pipeline to production inference without re-implementing it, while being explicit about which parts of that pipeline currently can and cannot make that trip.

State of the field

scikit-learn (Pedregosa et al., 2011) defines the fit/transform estimator contract that Gators reuses directly, and its preprocessing module and ColumnTransformer cover a subset of what Gators implements, but operate on NumPy (Harris et al., 2020)/pandas and are

single-threaded by default. `feature-engine` (Galli, 2021) extends `scikit-learn` with a broader catalogue of encoders, imputers, and discretizers on the same pandas-based footing. `category_encoders` (McGinnis et al., 2018) offers a further, encoder-focused catalogue with the same estimator contract. `Gators` overlaps in scope with all three but differs in its execution engine — `Polars`, a Rust-implemented (Matsakis & Klock, 2014), multi-threaded `DataFrame` library built on the Apache Arrow columnar format (Apache Software Foundation, 2016) with a lazy query optimizer (Vink & Polars Contributors, 2020) — and in shipping a built-in export path to a portable inference graph. `sklearn-onnx` (ONNX Contributors, 2019b) converts individual `scikit-learn` estimators to ONNX one at a time; `Gators` instead exports an entire fitted pipeline as one self-contained, checker-validated ONNX `ModelProto`. `TensorFlow Transform` (Baylor et al., 2017) takes the opposite approach to training/serving consistency, compiling preprocessing directly into a `TensorFlow` serving graph; `Gators` keeps training-time execution framework-agnostic (`Polars`) and generates a portable graph only at export time, at the cost of a documented, principled boundary on which transformers can currently make that trip (Section “Software design”).

Software design

Every transformer subclasses a common `_BaseTransformer`, combining `Pydantic`'s (Colvin & Pydantic Contributors, 2017) `BaseModel` (typed, validated, keyword-only configuration) with `scikit-learn`'s `BaseEstimator/TransformerMixin` (`get_params`, `set_params`, pipeline compatibility). `fit(X, y=None)` computes and stores statistics as private attributes and returns `self`; `transform(X)` applies those statistics and returns a new `Polars DataFrame`. A shared `__init_subclass__` hook gives every transformer consistent `LazyFrame` materialization, a `NotFittedError` guard, and automatic input-column/dtype bookkeeping, so that 107 independently authored transformers share one calling convention without reimplementing it individually. Internally, transformers batch column-wise operations into a single `Polars` expression list per `with_columns/select` call rather than looping and re-assigning the `DataFrame` column by column, which is the direct mechanism behind the library's measured performance advantage over pandas-based tooling: it lets one transform call compile to one query plan that the engine can parallelize across columns and cores, instead of many sequential ones. `gators.pipeline.Pipeline` chains named steps exactly as `sklearn.pipeline.Pipeline` does, remaining a valid `scikit-learn` estimator to downstream tooling. `gators.onnx_converters.pipeline_to_onnx` walks a fitted pipeline and, for each step, invokes a per-transformer-family ONNX converter; the resulting graph is opset-selected automatically and validated with `onnx.checker` before being returned. All data-cleaning, clipping, encoding, imputation, discretization, scaling, and numeric/datetime feature-generation transformers currently convert. Nine string-feature transformers do not — seven because they rely on variable-length tokenization, fuzzy edit distance, or regex-group/list-aggregate semantics that ONNX's string-tensor operator set cannot express, and two because they require a string-length or string-slice operator absent from the standard opset — and the six feature-selection transformers are not yet wired into the exporter. `check_pipeline_onnx_compatibility` lets a user audit a pipeline for this boundary before attempting a production export, and the exporter itself supports both fail-fast and pass-through-unchanged behavior for unsupported steps, rather than failing opaquely.

Research impact statement

`Gators` was developed by PayPal's Payment Service Provider (PSP) Data Team to support fraud-detection and risk-modeling workflows, where preprocessing and feature-engineering cost is a first-order concern at the row counts typical of transaction data, and where a consistent training-to-serving path materially reduces the risk of silent model degradation from re-implementation drift. Its open, rerunnable benchmark suite (comparing `Gators` against `scikit-learn` and `feature-engine` across row counts, feature counts, missingness, cardinality, and thread configurations, and comparing native execution against the ONNX-

exported path across batch sizes) gives downstream researchers a reproducible reference point for preprocessing performance claims in tabular ML pipelines, rather than a single, uncontextualized number. By documenting precisely which transformer families can and cannot currently be represented in a portable inference graph, and why, the project also contributes a concrete characterization of where graph-based, statically-shaped inference formats reach their limits for tabular preprocessing — a boundary condition relevant to any research building ONNX-based deployment tooling for classical (non-neural) ML pipelines, not only to this library's own users. Gators is available at <https://github.com/paypal/Gators/>.

AI usage disclosure

This paper was initially drafted by the authors. Portions of this project's development were carried out with the assistance of an AI coding agent (GitHub Copilot, built on large language models), operating under direct human review and direction. This included: authoring and running the extended benchmark suite (`benchmarks/run_matrix_benchmarks.py`, `benchmarks/run_onnx_benchmarks.py`); and helping expand and revise sections of the authors' draft, including this paper. All AI-assisted code changes were reviewed, tested against the existing 2,323-test suite, and benchmarked by a human author before being retained.

Acknowledgements

We thank Prem Thangamani for his managerial support of the development of Gators, PayPal's PSP Data Team for supporting the project more broadly, and the maintainers of Polars, scikit-learn, Pydantic, and ONNX, whose libraries this project builds directly on.

Apache Software Foundation. (2016). *Apache arrow: A cross-language development platform for in-memory data*. <https://arrow.apache.org>.

Baylor, D., Breck, E., Cheng, H.-T., Fiedel, N., Foo, C. Y., Haque, Z., Haykal, S., Ispir, M., Jain, V., Koc, L., Koo, C. Y., Lew, L., Mewald, C., Modi, A. N., Polyzotis, N., Ramesh, S., Roy, S., Whang, S. E., Wicke, M., ... Zinkevich, M. (2017). TFX: A TensorFlow-based production-scale machine learning platform. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 1387–1395. <https://doi.org/10.1145/3097983.3098021>

Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. *IEEE International Conference on Big Data (Big Data)*, 1123–1132. <https://doi.org/10.1109/BigData.2017.8258038>

Colvin, S., & Pydantic Contributors. (2017). *Pydantic: Data validation using Python type hints*. <https://docs.pydantic.dev>.

Galli, S. (2021). *Feature_engine: A Python package for feature engineering for machine learning*. *Journal of Open Source Software*, 6(65), 3642. <https://doi.org/10.21105/joss.03642>

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>

Matsakis, N. D., & Klock, F. S. (2014). The Rust language. *ACM SIGAda Ada Letters*, 34(3), 103–104. <https://doi.org/10.1145/2692956.2663188>

McGinnis, W., Siu, C., Andre, S., & Huang, H. (2018). *Category_encoders: A scikit-learn-contrib package of transformers for encoding categorical data*. https://github.com/scikit-learn-contrib/category_encoders.

ONNX Contributors. (2019a). *ONNX: Open neural network exchange*. <https://onnx.ai>.

- 139 ONNX Contributors. (2019b). *Sklearn-onnx: Convert scikit-learn models to ONNX*. <https://github.com/onnx/sklearn-onnx>.
140
- 141 Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M.,
142 Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,
143 Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python.
144 *Journal of Machine Learning Research*, 12, 2825–2830.
- 145 Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost:
146 Unbiased boosting with categorical features. *Advances in Neural Information Processing*
147 *Systems (NeurIPS)*, 31.
- 148 Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young,
149 M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning
150 systems. *Advances in Neural Information Processing Systems (NeurIPS)*, 28.
- 151 Vink, R., & Polars Contributors. (2020). *Polars: Fast multi-threaded, hybrid-out-of-core*
152 *DataFrame library*. <https://www.pola.rs>.

DRAFT